Strings (UNIT 3)

Strings:

In C programming, a string is a sequence of characters terminated with a null character `\0`. For example:

```
char c[] = "c string";
```

When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character `\0` at the end by default.

c		s	t	r	i	n	g	\0

How to declare a string?

Here's how you can declare strings:

Declaration of strings: Declaring a string is as simple as declaring a one dimensional array. Below is the basic syntax for declaring a string.

```
char str_name[size];
```

```
char s[5];
```

s[0]	s[1]	s[2]	s[3]	s[4]

How to initialize strings:

You can initialize strings in a number of ways.

```
char c[] = "abcd";

char c[50] = "abcd";

char c[] = {'a', 'b', 'c', 'd', '\0'};

char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

Let's take another example:

```
char c[5] = "abcde";
```

Here, we are trying to assign 6 characters (the last character is '\0') to a char array having 5 characters. This is bad and you should never do this.

Read String from the user

You can use the scanf() function to read a string.

The scanf() function reads the sequence of characters until it encounters whitespace (space, newline, tab etc.).

Example 1: scanf() to read a string

```
#include <stdio.h>
int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

Output

```
Enter name: Dennis Ritchie
Your name is Dennis.
```

Even though Dennis Ritchie was entered in the above program, only "Ritchie" was stored in the name string. It's because there was a space after Dennis.

gets() and puts()

Functions gets() and puts() are two string functions to take string input from the user and display it respectively.

```
#include<stdio.h>
```

```
int main()
{
    char name[30];
    printf("Enter name: ");
    gets(name);     //Function to read string from user.
    printf("Name: ");
    puts(name);    //Function to display string.
    return 0;
}
```

Note: Though, gets() and puts() function handle strings, both these functions are defined in "stdio.h" header file.

String Manipulations In C Programming Using Library Functions

You need to often manipulate strings according to the need of a problem. Most, if not all, of the time string manipulation can be done manually but, this makes programming complex and large.

To solve this, C supports a large number of string handling functions in the standard library "string.h".

Few commonly used string handling functions are discussed below:

Function	Work of Function
strlen()	computes string's length

Function	Work of Function
strcpy()	copies a string to another
strcat()	concatenates(joins) two strings
strcmp()	compares two strings

C strlen()

The strlen() function calculates the length of a given string.

The strlen() function takes a string as an argument and returns its length.

The returned value is of type long int.

It is defined in the <string.h> header file.

Example: C strlen() function

```
#include <stdio.h>
#include <string.h>
int main()
{
    char a[20]="Program";
```

```
    char b[20]={'P','r','o','g','r','a','m','\0'};

    printf("Length of string a = %ld \n",strlen(a));
    printf("Length of string b = %ld \n",strlen(b));

    return 0;
}
```

Output

```
Length of string a = 7
Length of string b = 7
```

Note that the strlen() function doesn't count the null character \0 while calculating the length.

C strcpy()

The strcpy() function copies the string to the another character array.

strcpy() Function prototype

```
char* strcpy(char* destination, const char* source);
```

The strcpy() function copies the string pointed by source (including the null character) to the character array destination.

The function also returns the copied array.

The strcpy() function is defined in the string.h header file.

Example: C strcpy()

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[10]= "awesome";
    char str2[10];
    char str3[10];

    strcpy(str2, str1);
    strcpy(str3, "well");
    puts(str2);
    puts(str3);

    return 0;
```

```
}
```

Output

```
awesome
well
```

It is important to note that the destination array should be large enough to copy the array. Otherwise, it may result in undefined behavior.

C strcat()

The function strcat() concatenates two strings.

In C programming, strcat() concatenates (joins) two strings.

The strcat() function is defined in <string.h> header file.

C strcat() Prototype

```
char *strcat(char *dest, const char *src)
```

It takes two arguments, i.e, two strings or character arrays, and stores the resultant concatenated string in the first string specified in the argument.

The pointer to the resultant string is passed as a return value.

Example: C strcat() function

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[] = "This is ", str2[] = "a string";

    //concatenates str1 and str2 and resultant string is stored in str1.
    strcat(str1,str2);

    puts(str1);
    puts(str2);

    return 0;
}
```

Output

```
This is a string

a string
```

C strcmp()

The strcmp() function compares two strings and returns 0 if both strings are identical.

C strcmp() Prototype

```
int strcmp (const char* str1, const char* str2);
```

The strcmp() function takes two strings and returns an integer.

The strcmp() compares two strings character by character.
If the first character of two strings is equal, the next character of two strings are compared. This continues until the corresponding characters of two strings are different or a null character '\0' is reached.
It is defined in the string.h header file.

Return Value from strcmp()

Return Value	Remarks
0	if both strings are identical (equal)
Negative	if the ASCII value of the first unmatched character is less than second.
positive integer	if the ASCII value of the first unmatched character is greater than second.

Example: C strcmp() function

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "abcd", str2[] = "abCd", str3[] = "abcd";
    int result;

    // comparing strings str1 and str2
    result = strcmp(str1, str2);
    printf("strcmp(str1, str2) = %d\n", result);
```

```
    // comparing strings str1 and str3
    result = strcmp(str1, str3);
    printf("strcmp(str1, str3) = %d\n", result);

    return 0;
}
```

Output

```
strcmp(str1, str2) = 32

strcmp(str1, str3) = 0
```

The first unmatched character between string str1 and str2 is third character. The ASCII value of 'c' is 99 and the ASCII value of 'C' is 67. Hence, when strings str1 and str2 are compared, the return value is 32. When strings str1 and str3 are compared, the result is 0 because both strings are identical.

C program to find the length of a String without using function strlen()

In the following C program we are counting the number of characters in a given String to find out and display its length on console. Upon execution of this program, the user would be asked to enter a string, then the program would count the chars and would display the length of input string as output.

C Program – finding length of a String without using

standard library function strlen

```
/* C Program to find the length of a String without
 *  using any standard library function
 */
#include <stdio.h>
int main()
{
    /* Here we are taking a char array of size
     * 100 which means this array can hold a string
     * of 100 chars. You can change this as per requirement
     */
    char str[100],i;
    printf("Enter a string: \n");
    scanf("%s",str);

    // '\0' represents end of String
    for(i=0; str[i]!='\0'; ++i);
       printf("\nLength of input string: %d",i);

    return 0;
}
```

Output:

```
Enter a string:
Chaitanya

Length of input string: 9
```

C program to copy string without using strcpy()

function

We can use inbuilt strcpy() function to copy one string to other but here, this program copies the content of one string to another manually without using strcpy() function.
Approach : Here we are giving one string in input and then with the help of **for loop** we transfer the content of first array to the second array.
Error : If destination string length is less than source string, entire string value won’t be copied into destination string.
For example, consider destination string length is 20 and source string length is 30. Then, only 20 characters from source string will be copied into destination and remaining 10 characters will be truncated.

```
#include <stdio.h>
int main()
{
    // s1 is the source( input) string and s2 is the destination string
    char s1[] = "Hello!", s2[100], i;

    // Print the string s1
    printf("string s1 : %s\n", s1);

    // Execute loop till null found
    for (i = 0; s1[i] != '\0'; ++i) {
        // copying the characters by
        // character to str2 from str1
        s2[i] = s1[i];
    }

    s2[i] = '\0';

    // printing the destination string
    printf("String s2 : %s", s2);

    return 0;
```

}
Output:

```
string s1 : Hello!

String s2 : Hello!
```

C Program to concatenate two strings without using strcat

In the following program user would be asked to enter two strings and then the program would concatenate them. For concatenation we have not used the standard library function strcat(), instead we have written a logic to append the second string at the end of first string.

C program for string concatenation

```
/* C program to concatenate two strings without
 * using standard library function strcat()
 */
#include <stdio.h>
int main()
{
   char str1[50], str2[50], i, j;
   printf("\nEnter first string: ");
   scanf("%s",str1);
   printf("\nEnter second string: ");
   scanf("%s",str2);
   /* This loop is to store the length of str1 in i
    * It just counts the number of characters in str1
    * You can also use strlen instead of this.
    */
   for(i=0; str1[i]!='\0'; ++i);

   /* This loop would concatenate the string str2 at
    * the end of str1
```

```
   */
  for(j=0; str2[j]!='\0'; ++j, ++i)
  {
     str1[i]=str2[j];
  }
  // \0 represents end of string
  str1[i]='\0';
  printf("\nOutput: %s",str1);

  return 0;
}
```

Output:

Program to Compare Two Strings without using strcmp( ) in C

```
#include<stdio.h>

main()

{



        char string1[5],string2[5];

        int i,temp = 0;
```

```
printf("Enter the string1 value: ");
gets(string1);

printf(" Enter the String2 value: ");
gets(string2);

for(i=0; string1[i]!='\0'; i++)
{
    if(string1[i] == string2[i])
    temp = 1;
    else
    temp = 0;
}

if(temp == 1)
printf("Both strings are same.");

else

printf("Both string not same.");
```

```
getch();

}
```

```
Enter 1st string :   Studytonight

Enter 2nd string :   studytonight

Both the strings are different.
```